Pengolahan Citra

Kompresi Citra dan Reduksi Data

Ken Ratri, MT
Ken_ratri@ithb.ac.id

Topik

1. Fundamentals
2. Model Kompresi citra
3. Basic Metode Kompresi

Kendala Data Citra Dijital

- Mengapa perlu kompresi dan reduksi data?
  - Data citra umumnya berukuran besar
  - Tidak praktis dalam aspek penyimpanan, proses dan transmisi
  - Perlu reduksi atau pemampatan data dengan mengurangi *redundancy* atau duplikasi data
- Data redundancy:
  - adalah bagian data yang tidak mengandung informasi terkait atau merupakan pengulangan dari informasi yang sudah dinyatakan sebelumnya atau sudah diketahui

Contoh Aplikasi

- Aplikasi yang membutuhkan image compression: dimana perkembangannya ditentukan oleh efisiensi pada manipulasi data, penyimpanan, dan transmisi citra biner / monokrom / berwarna:
  - Televideo-conferencing
  - Remote sensing
  - Telemedical / Medical imaging
  - Facsimile transmission

Kategori Teknik Kompresi Citra

- Information preserving (lossless compression): teknik yang memproses data asli menjadi bentuk yang lebih ringkas tanpa hilangnya informasi. Contoh: Aplikasi biomedis, SIG.
- Lossy compression: teknik mendapatkan data yang lebih ringkas dengan melalui suatu proses penghampiran (approksimasi) dari data asli dengan tingkat error yang dapat diterima. Contoh: TV broadcast, teleconference.

Ex

- Data movie 2 jam SD video : 720 x 480 x 24 bit/px
- Sekuensial frame 30 fps :

$$30\ fps \text{ x } (720 \text{ x } 480)\text{px/frame x } 3 \text{ bytes/px} = 31.104.000 \text{ bytes/sec}$$

2 jam movie:

$$31.104.000 \text{ bytes/sec x } (60^2)\text{sec/hr x } 2 \text{ hrs} \cong 2.24 \text{ x } 10^{11}\ bytes$$

Konsep *Data Redundancy*

- Tidak abstrak dan dapat dinyatakan dalam entitas besaran matematis
- Relative data redundancy RD:

$$R_D = 1 - \frac{1}{C_R} \quad dan \quad C_R = \frac{n_1}{n_2}$$

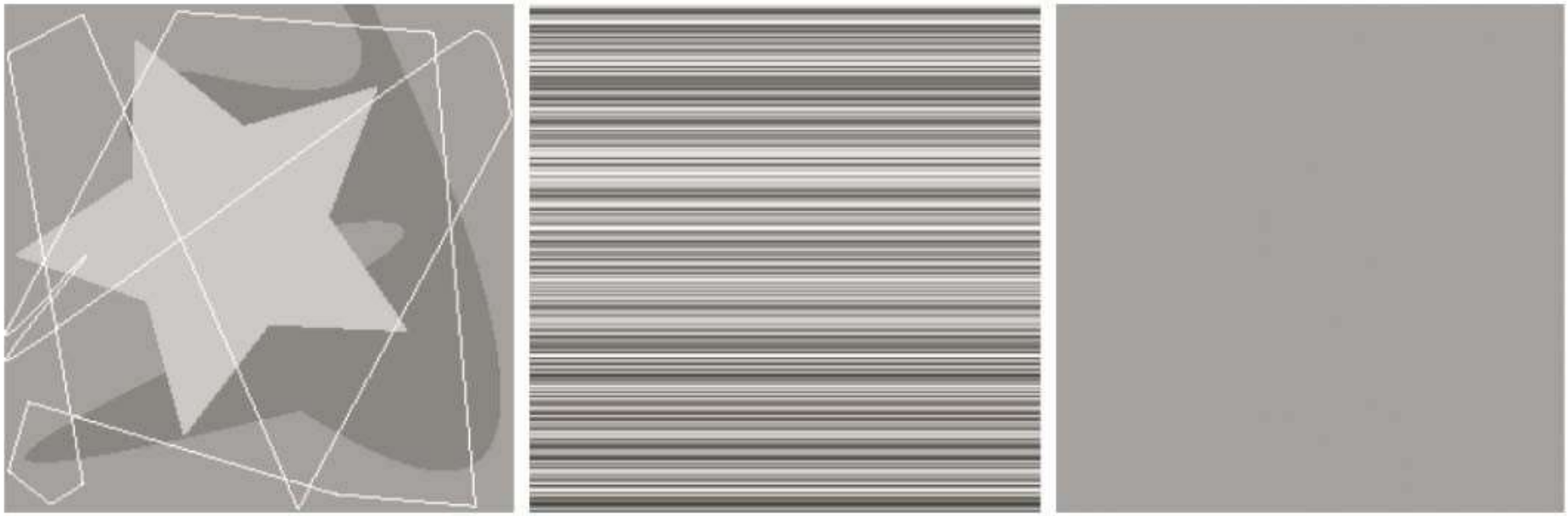
n1 dan n2: jumlah satuan informasi yang dibawa data set 1 dan data set 2.

- Bila n2 = n1, maka CR = 1 dan RD = 0 (tidak ada redundancy).
- Bila n2 << n1, maka CR mendekati tak terhingga dan RD mendekati 1 (terjadi redundancy yang tinggi).
- Bila n2 >> n1, maka CR (ratio kompresi) mendekati 0 dan RD mendekati minus tak terhingga (informasi yang dibawa data set 2 jauh lebih besar dari data set 1)
- Bila 0 < CR < 1 dan RD rendah (tingkat redundancy rendah).

Data Redundancy

- Ada 3 jenis data redundancy pada citra:
  1. Coding redundancy ; jumlah bit lebih sedikit direpresentasikan dengan simbol frekuensi (huruf, angka, bit). Setiap pixel ditetapkan sebagai sekuens dari *code symbols(code word ; lenght).*
  2. Spatial dan temporal redudancy ; Interpixel redundancy, korelasi spatian-- pixel bertetangga memiliki nilai yang sama
  3. Irrelevant information; Psychovisual redundancy, sistem visual manusia tidak dapat secara simultan membedakan semua warna

Suatu data dikatakan terkompresi jika satu atau lebih redundancy tersebut bisa dikurangi atau dihilangkan.



a b c

FIGURE 8.1 Computer generated 256 × 256 × 8 bit images with (a) coding redundancy, (b) spatial redundancy, and (c) irrelevant information. (Each was designed to demonstrate one principal redundancy but may exhibit others as well.)

Coding redundancy (1)

- Variabel r_k dalam interval [0, L-1] representasi intensitan M x N, memiliki probabilitas :

$$p_r(r_k) = \frac{n_k}{MN} \quad \text{k} = 0,1,2,3,....\text{L} - 1$$

- Rata-rata bit untuk setiap pixel :

$$L_{avg} = \sum_{\text{k}=0}^{\text{L-1}} \text{l}_{(\text{rk})} p_r(r_k)$$

- Example of variable length coding
  - 8 discrete gray levels in an image, with distribution given by

r_k	$p_r(r_k)$	Code 1	$l_1(r_k)$	Code 2	$l_2(r_k)$
$r_0 = 0$	0.19	000	3	11	2
$r_1 = \frac{1}{7}$	0.25	001	3	01	2
$r_2 = \frac{2}{7}$	0.21	010	3	10	2
$r_3 = \frac{3}{7}$	0.16	011	3	001	3
$r_4 = \frac{4}{7}$	0.08	100	3	0001	4
$r_5 = \frac{5}{7}$	0.06	101	3	00001	5
$r_6 = \frac{6}{7}$	0.03	110	3	000001	6
$r_7 = 1$	0.02	111	3	000000	6

  - For code 1, L_{avg} is 3 bits
  - For code 2, L_{avg} is given by

$$\begin{aligned} L_{\text{avg}} &= \sum_{k=0}^{7} l_2(r_k) p_r(k) \\ &= 2(0.19) + 2(0.25) + 2(0.21) + 3(0.16) + 4(0.08) + 5(0.06) + 6(0.03) + 6(0.02) \\ &= 2.7\text{bits} \end{aligned}$$

  - Compression ratio $C_R = \frac{3}{2.7} = 1.11$
  - Redundancy $R_D = 1 - \frac{1}{1.11} = 0.099$

Spatial/Temporal Redundancy

garis intensitas konstan pada Gambar page 9B

- Semua intensitas 256 dengan kemungkinan sama; histogram uniform
- Gambar tidak dapat dikompresi oleh variabel length encoding
- Dalam hal ini, fixed length code, meminimalkan jumlah bit yang diperlukan untuk gambar
- piksel independen satu sama lain dalam arah vertikal
- setiap baris yang identik, piksel maksimal berkorelasi (tergantung pada satu sama lain) dalam arah horisontal

Spatial/Temporal Redundancy

- redundansi spasial yang signifikan diminimalkan dengan run-length pairs atau pemetaan
- Pasangan run-length menggambarkan awal dari intensitas baru dan jumlah piksel dengan intensitas berturut-turut
- Untuk redundansi temporal, dapat menghitung perbedaan dari satu frame ke depan dan $_{\gamma}$ menggunakannya sebagai kode
- data redundancy adalah korelasi antar piksel dimana intensitas suatu piksel dapat diperkirakan dari intensitas piksel-piksel tetangganya.
- informasi yang dibawa oleh setiap piksel relatif tidak besar atau kontribusi setiap piksel kepada gambar secara keseluruhan adalah redundan.

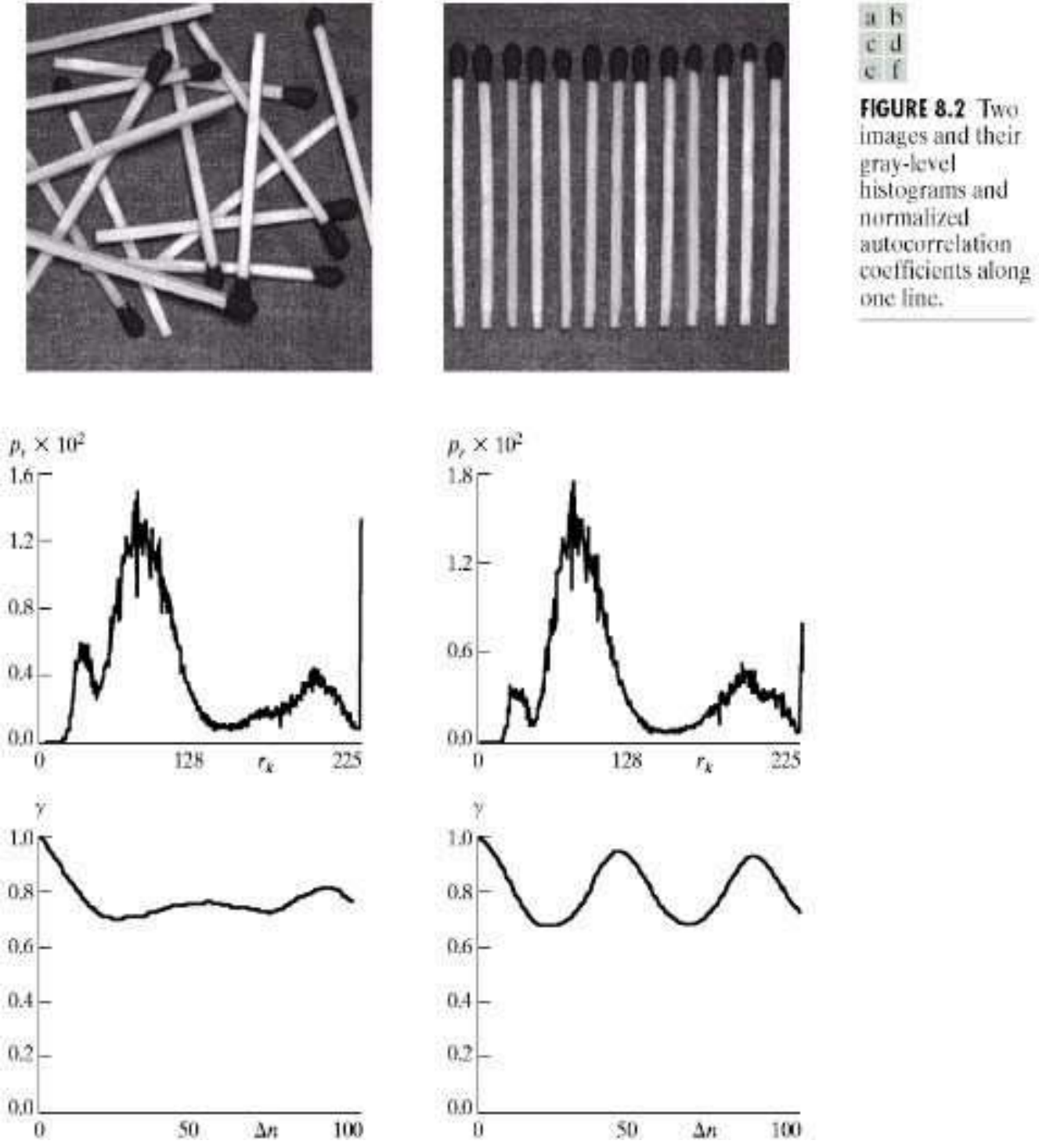


FIGURE 8.2 Two images and their gray-level histograms and normalized autocorrelation coefficients along one line.

Spatial/Temporal Redundancy**(2)**

- Citra yang mengandung interpixel redundancy dapat direpresentasi ke dalam bentuk yang lebih efisien (non-visual format).
- Dimana citra tidak dinyatakan dalam bentuk matriks dari intensitas piksel-pikselnya, akan tetapi dipetakan (mapping) dalam bentuk perbedaan intensitas antar piksel yang bersebelahan.
- Bentuk data yang dimampatkan ini bersifat reversible, artinya dapat direkonstruksi kembali menjadi citra asalnya.
- Run-Length Coding: (g1,R1), (g2,R2) … g_i menyatakan tingkat keabuan dan Ri menyatakan jumlah piksel bersebelahan pada baris bersangkutan yang mempunyai intensitas gi.

Psychovisual Redundancy (1)

Mach Band

- Suatu fenomena dimana intensitas keabuan yang bervariasi dilihat sebagai intensitas konstan, artinya: mata tidak mempunyai sensitivitas yang sama terhadap semua informasi yang dianggap penting dan tidak penting.
- Hal tersebut diatas dapat dikatakan sebagai psychovisual redundancy, yang bila dihilangkan tidak mengganggu persepsi kwalitas citra.

Psychovisual Redundancy (2)

- **Quantization:**
  - Eliminasi mengakibatkan kerugian informasi bersifat kualitas. Pemetaan dari daerah intensitas yang lebar menjadi daerah intensitas terbatas (batas kemampuan visual manusia)
  - Mata sangat sensitif terhadap informasi edge / garis batas / patahan. Proses kwantisasi dapat disesuaikan untuk mempertahankan informasi edge seperti yang dilakukan pada Improved Gray Scale (IGS) Quantization.
- **Irreversible Operation:**
  - Quantization mengakibatkan hilangnya sebagian informasi visual, dengan demikian teknik ini termasuk pada kategori lossy data compression.

Contoh

a b c

FIGURE 8.4
(a) Original image.
(b) Uniform quantization to 16 levels. (c) IGS quantization to 16 levels.



Psychovisual Redundancy (4)

- Lossy compression → perlu adanya kriteria penilaian untuk mengatakan apakah hasil kompresi bagus atau tidak.
- Kriteria penilaian terbagi 2:
  - Secara objektif

    Hilangnya informasi dinyatakan sebagai fungsi dari citra input ke dalam citra output
  - Secara subjektif

    Berdasarkan pada penilaian mata manusia.

    Tingkatannya: Excellent, fine, passable, marginal, inferior, unusable.

Psychovisual Redundancy (5)

- Kesalahan akibat hilangnya informasi:

  f(x,y) adalah citra asli dan f'(x,y) adalah citra approksimasi

  Total Error:

$$Error = \sum_{x=0}^{M-1}\sum_{y=0}^{N-1}(f'(x,y) - f(x,y))$$

  Root-Mean-Square Error:

$$E_{RMS} = \left(\frac{1}{MN}\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}(f'(x,y) - f(x,y))^2\right)^{1/2}$$

  Mean-square Signal-to-Ratio:

$$SNR_{MS} = \frac{\sum_{x=0}^{M-1}\sum_{y=0}^{N-1} f'(x,y)^2}{\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}(f'(x,y) - f(x,y))^2}$$

Penilaian Subjektif

Value	Rating	Description
1	Excellent	An image of extremely high quality, as good as you could desire.
2	Fine	An image of high quality, providing enjoyable viewing. Interference is not objectionable.
3	Passable	An image of acceptable quality. Interference is not objectionable.
4	Marginal	An image of poor quality; you wish you could improve it. Interference is somewhat objectionable.
5	Inferior	A very poor image, but you could watch it. Objectionable interference is definitely present.
6	Unusable	An image so bad that you could not watch it.

Basic Metode Kompresi

- Kuantisasi
- Run-length coding
- Hufman Coding
- Dictionary Based/LZW ; Lempel–Ziv–Welch
- Arithmetic Coding

Kompresi Berbasis Kuantisasi

- Menggunakan metode pengurangan jumlah intensitas warna, sehingga dapat mengurangi jumlah bit yang digunakan untuk merepresentasikan citra.
- Bersifat *lossy*
- Di kompresi dengan merubah derajat keabuan, misalnya dari 4 derajat keabuan menjadi 2
- Algoritma metode kuantisasi:
    1. Buat **histogram citra** semula (**citra** yang akan dimampatkan).
    2. Identifikasi *n* buah kelompok di dalam **histogram** sedemikian sehingga setiap kelompok mempunyai kira-kira *P*/*n* buah *pixel*.
    3. Nyatakan setiap kelompok dengan derajat keabuan 0 sampai *n* –1. Setiap *pixel* di dalam kelompok dikodekan kembali dengan nilai derajat keabuan yang baru.

Contoh gambar berukuran 5 × 13 *pixel,* yang akan dimampatkan menjadi citra dengan 4 derajat keabuan (0 s/d 3), jadi setiap derajat keabuan direpresentasikan dengan 2 bit.

2 9 6 4 8 2 6 3 8 5 9 3 7

3 8 5 4 7 6 3 8 2 8 4 7 3

3 8 4 7 4 9 2 3 8 2 7 4 9

3 9 4 7 2 7 6 2 1 6 5 3 0

2 0 4 3 8 9 5 4 7 1 2 8 3

k	n_k
0	2
1	2
2	9
3	11
4	9
5	4
6	5
7	8
8	9
9	6

Ada 65 *pixel*, dikelompokkan menjadi 4 kelompok derajat keabuan. Tiapkelompok ada sebanyak rata-rata 65/4 = 16.25 *pixel* per kelompok:

k	n_k	n_b	k_b
0	2	13	0
1	2		
2	9		
3	11	20	1
4	9		
5	4	17	2
6	5		
7	8		
8	9	15	3
9	6		

Ukuran **citra** sebelum pemampatan (1 derajat keabuan = 4 bit):

65 × 4 bit = 260 bit

Ukuran **citra** setelah pemampatan (1 derajat keabuan = 2 bit):

65 × 2 bit = 130 bit

Ratio = 100% -130/ 260 X 100 = 50%

Run-length coding

- Contoh Citra:

0 0 0 0 1 1 0 0
0 0 0 1 1 1 1 0
0 0 0 1 1 1 1 0
0 0 1 1 1 1 1 0
0 0 1 1 1 1 1 0
0 0 0 1 1 1 0 0
0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 0

Run-Length Code Representation:

(0,4), (1,2), (0,2)
(0,3), (1,4), (0,1)
(0,3), (1,4), (0,1)
(0,2), (1,5), (0,1)
(0,2), (1,5), (0,1)
(0,3), (1,3), (0,3)
(0,4), (1,1), (0,3)
(0,8)

- Informasi Yang Disimpan:

22 pairs of integers

Contoh 10.2. [LOW91] Tinjau citra 10 × 10 *pixel* dengan 8 derajat keabuan yang dinyatakan sebagai matriks derajat keabuan sebagai berikut

0	0	0	0	0	2	2	2	2	2
0	0	0	1	1	1	1	2	2	2
1	1	1	1	1	1	1	1	1	1
4	4	4	4	3	3	3	3	2	2
3	3	3	5	5	7	7	7	7	6
2	2	6	0	0	0	0	1	1	0
3	3	4	4	3	2	2	2	1	1
0	0	0	0	0	0	0	0	1	1
1	1	1	1	0	0	0	2	2	2
3	3	3	2	2	2	1	1	1	1

semuanya ada 100 buah nilai.

Pasangan nilai untuk setiap baris *run* yang dihasilkan dengan metode pemampatan RLE:

(0, 5), (2, 5)
(0, 3), (1, 4), (2, 3)
(1, 10)
(4, 4), (3, 4), (2 2)
(3, 3), (5, 2), (7, 4), (6, 1)
(2, 2), (6, 1), (0, 4), (1, 2), (0, 1)
(3, 2), (4, 2), (3, 1), (2, 2), (1, 2)
(0, 8), (1, 2)
(1, 4), (0, 3), (2, 3)
(3, 3), (2, 3), (1, 4)

semuanya ada 31 pasangan nilai atau 31 × 2 = 62 nilai.

Ukuran citra sebelum pemampatan (1 derajat keabuan = 3 bit) adalah 100 × 3 bit = 300 bit, sedangkan ukuran citra setelah pemampatan (derajatk keabuan = 3 bit, *run length* = = 4 bit):

$$(31 \times 3) + (31 \times 4) \text{ bit} = 217 \text{ bit}$$

Nisbah pemampatan = $(100\% - \frac{217}{300} \times 100\%) = 27.67\%$, yang artinya 27.67% dari citra semula telah dimampatkan. ■

Versi lain dari metode *RLE* adalah dengan menyatakan seluruh baris citra menjadi sebuah baris *run*, lalu menghitung *run-length* untuk setiap derajat keabuan yang berurutan. Sebagai contoh, tinjau sebuah citra sebagai berikut:

1	2	1	1	1	1
1	3	4	4	4	4
1	1	3	3	3	5
1	1	1	1	3	3

Nyatakan sebagai barisan nilai derajat keabuan:

1 2 1 1 1 1 3 4 4 4 4 1 1 3 3 3 5 1 1 1 1 3 3

semuanya ada 24 nilai.

Pasangan nilai dari *run* yang dihasilkan dengan metode pemampatan *RLE*:

(1, 1) (2, 1) (1, 5) (3, 1) (4, 4) (1, 2) (3, 3) (5, 1) (1, 4) (3, 2)

Hasil pengkodean:

1 1 2 1 1 5 3 1 4 4 1 2 3 3 5 1 1 4 3 2

semuanya ada 20 nilai. Jadi, kita sudah menghemat 4 buah nilai.

Huffman

- histogram gray-level dari sebuah citra dapat pula menyediakan sebuah pengertian besar kedalam pengkonstruksian kode untuk mengurangi jumlah data yang digunakan untuk menampilkan citra itu.
- Terjadi bila suatu kode simbol yang digunakan terdiri dari sejumlah bit yang melebihi jumlah bit yang diperlukan untuk representasi setiap simbol (dalam hal ini: tingkat keabuan piksel citra).
- Teknik eliminasinya dengan cara mengubah pengkodean.Tingkat keabuan yang probabilitas terjadinya tinggi diberi panjang kode yang pendek, dan bila sebaliknya diberi kode yang panjang

Original source			Source reduction							
Symbol	Probability	Code	1		2		3		4	
a_2	0.4	1	0.4	1	0.4	1	0.4	1	0.6	0
a_6	0.3	00	0.3	00	0.3	00	0.3	00	0.4	1
a_1	0.1	011	0.1	011	0.2	010	0.3	01		
a_4	0.1	0100	0.1	0100	0.1	011				
a_3	0.06	01010	0.1	0101						
a_5	0.04	01011								

FIGURE 8.8 Huffman code assignment procedure.

Digunakan :
JPEG
MPEG
H.261
H.262
H.263
H. 264

Huffman

Langkah-langkah Algoritma *Huffman Coding* :

1. Urutkan menaik nilai intensitas berdasarkan probabilitas kemunculan (Pk=nk/n). Setiap nilai keabuan dinyatakan sebagai pohon bersimpul tunggal. Setiap simpul di-*assign* dengan frekuensi kemunculan nilai keabuan tersebut.
2. Gabung dua buah pohon yang mempunyai frekuensi kemunculan paling kecil pada sebuah akar. Akar mempunyai frekuensi yang merupakan jumlah dari frekuensi dua buah pohon penyusunnya.
3. Ulangi langkah 2 sampai tersisa hanya satu buah pohon biner
4. Beri label setiap sisi pada pohon biner. Sisi kiri dilabeli dengan 0 dan sisi kanan dilabeli dengan 1.
5. Telusuri pohon biner dari akar ke daun. Barisan label-label sisi dari akar ke daun menyatakan kode Huffman untuk derajat keabuan yang bersesuaian.

Contoh terdapat citra yang berukuran 64 × 64 dengan 8 derajat keabuan (*k*) dan jumlah seluruh *pixel* (*n*) = 64 × 64 = 4096

k	n_k	p(k) = n_k / n
0	790	0.19
1	1023	0.25
2	850	0.21
3	656	0.16
4	329	0.08
5	245	0.06
6	122	0.03
7	81	0.02

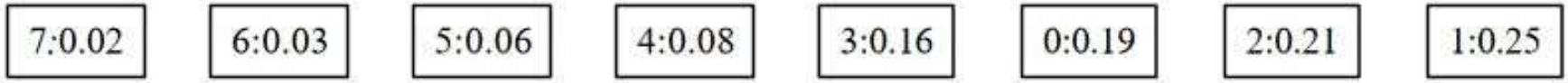
Ukuran citra sebelum kompresi : 4096 x 3 = 12288 bit

Setelah kompresi 11053 bit

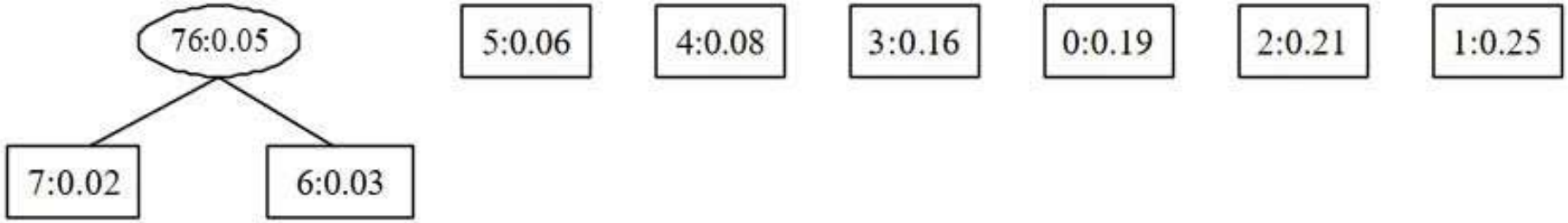
Ratio = 100% - 11053/12288 x 100% = 10%

$$rasiokompresi = 100\% - \left(\frac{ukurancitrahasilkompresi}{ukurancitrasemula} x100\% \right)$$

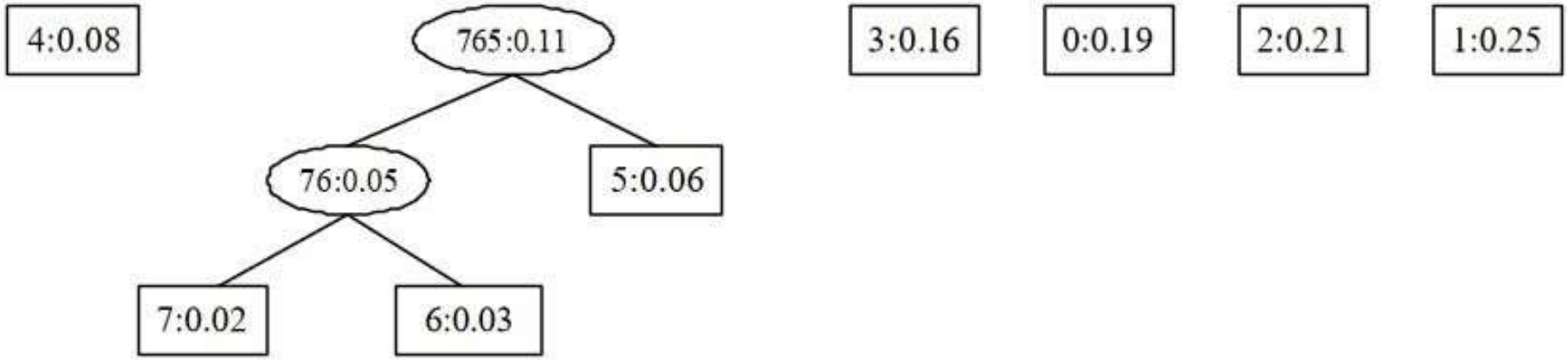
1.


7:0.02
6:0.03
5:0.06
4:0.08
3:0.16
0:0.19
2:0.21
1:0.25

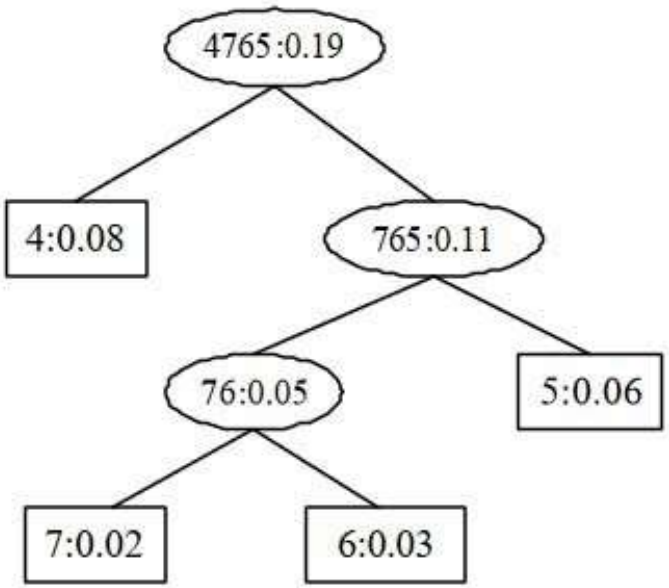
2.


76:0.05
5:0.06
4:0.08
3:0.16
0:0.19
2:0.21
1:0.25
7:0.02
6:0.03

3.

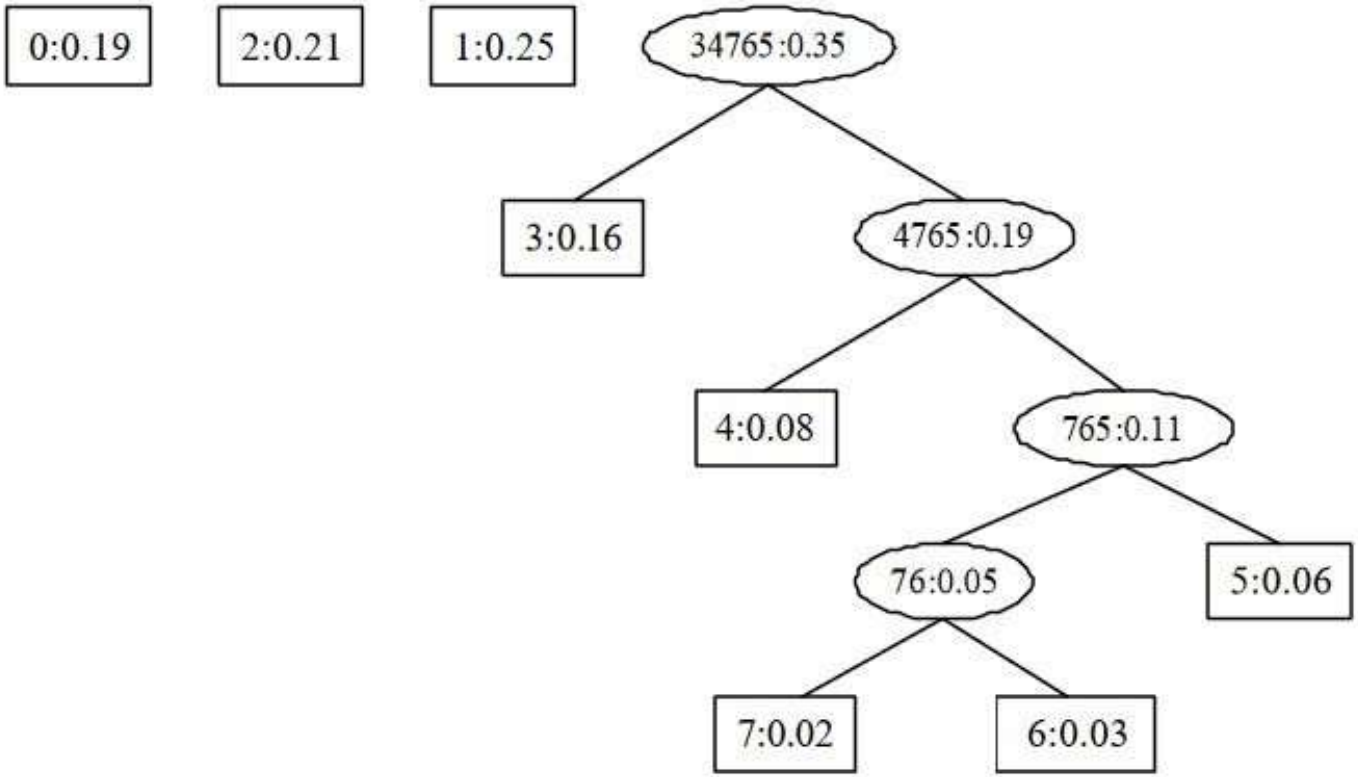

4:0.08
765:0.11
3:0.16
0:0.19
2:0.21
1:0.25
76:0.05
5:0.06
7:0.02
6:0.03

4.

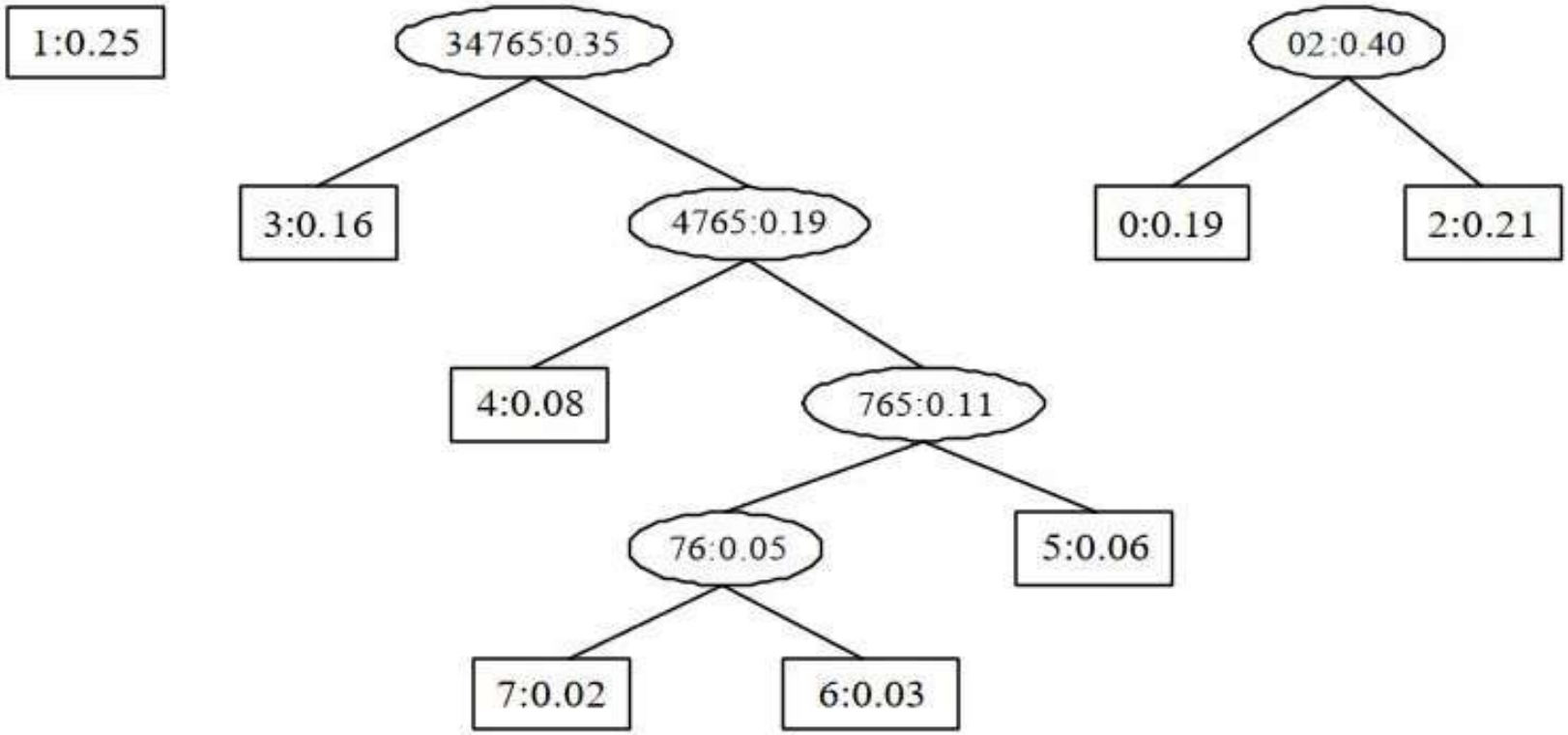


3:0.16 0:0.19 2:0.21 1:0.25

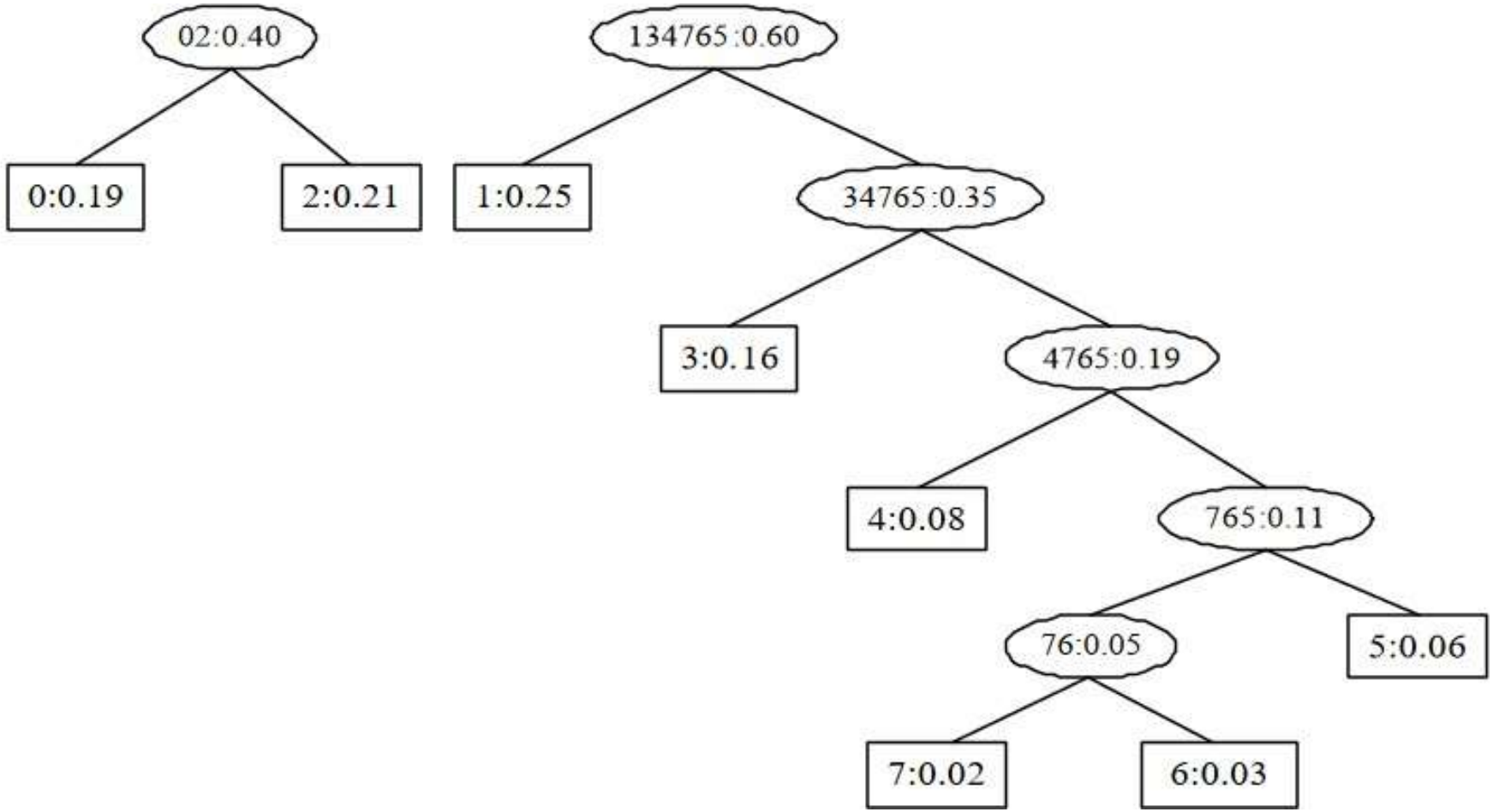
5.



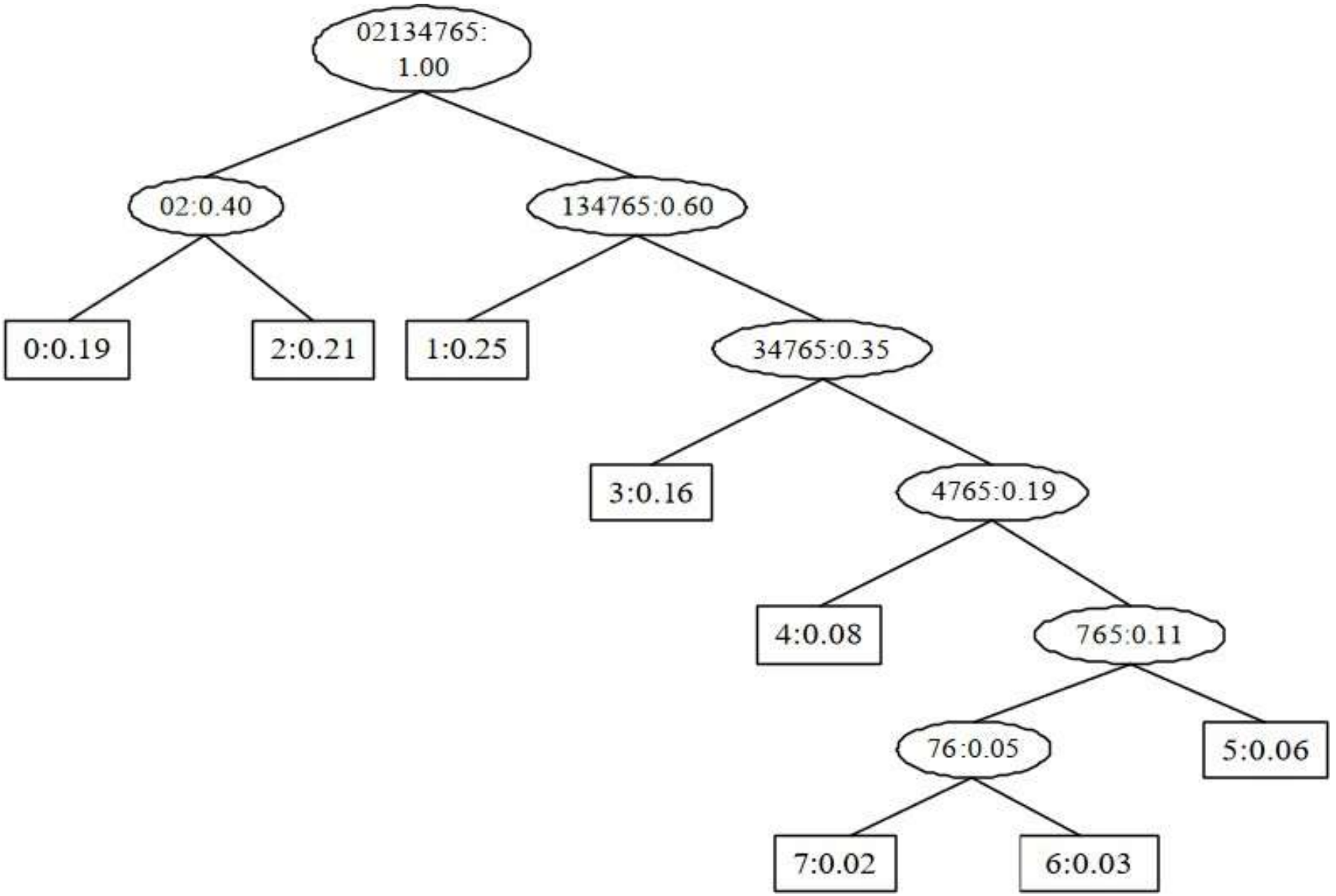
6.


1:0.25
34765:0.35
02:0.40
3:0.16
4765:0.19
0:0.19
2:0.21
4:0.08
765:0.11
76:0.05
5:0.06
7:0.02
6:0.03

7.


02:0.40
134765:0.60
0:0.19
2:0.21
1:0.25
34765:0.35
3:0.16
4765:0.19
4:0.08
765:0.11
76:0.05
5:0.06
7:0.02
6:0.03

8.


02134765:
1.00
02:0.40
134765:0.60
0:0.19
2:0.21
1:0.25
34765:0.35
3:0.16
4765:0.19
4:0.08
765:0.11
76:0.05
5:0.06
7:0.02
6:0.03

Algoritma Kompressi LZW

```
BEGIN
  s = next input character;
        while not EOF
        {
                c = next input character;
                if s + c exists in the diactionary
                     s = s + c
                   else
                  {
                Output the code for s;
                Add string s + c to the dictionary with a new code
                s = c;
                  }
           }
END
```

39	39	126	126
39	39	126	126
39	39	126	126
39	39	126	126

Currently Recognized Sequence	Pixel Being Processed	Encoded Output	Dictionary Location (Code Word)	Dictionary Entry
	39			
39	39	39	256	39-39
39	126	39	257	39-126
126	126	126	258	126-126
126	39	126	259	126-39
39	39			
39-39	126	256	260	39-39-126
126	126			
126-126	39	258	261	126-126-39
39	39			
39-39	126			
39-39-126	126	260	262	39-39-126-126
126	39			
126-39	39	259	263	126-39-39
39	126			
39-126	126	257	264	39-126-126
126		126		

TABLE 8.7 LZW coding example.

Algoritma Dekompresi

```
BEGIN
  S = NULL;
  while not EOF{
        K = NEXT INPUT CODE;
        Entry = dictionary entry for K;
        Ouput entry;
        if(s != NULL)
              add string s + entry[0] to dictionary with new code
        S = Entry;
  }
END
```

Arithmetic Coding

- The algorithm can be outlined as follows. Suppose we have a vocabulary of n symbols:
  1. Divide the interval [0,1) into n segments corresponding to the n symbols; the segment of each symbol has a length proportional to its probability. Each segment i has an upper bound U and lower bound L corresponding to the start of the segment and the end of the segment (U - L = Pi).
  2. Choose the segment that corresponds to the first symbol in the message string. This is the new current interval with its computed new upper and lower bounds.
  3. Divide the new current interval again into n new segments with length proportional to the symbols probabilities and compute the new intervals accordingly.

Source Symbol	Probability	Initial Subinterval
a_1	0.2	[0.0, 0.2)
a_2	0.2	[0.2, 0.4)
a_3	0.4	[0.4, 0.8)
a_4	0.2	[0.8, 1.0)

